



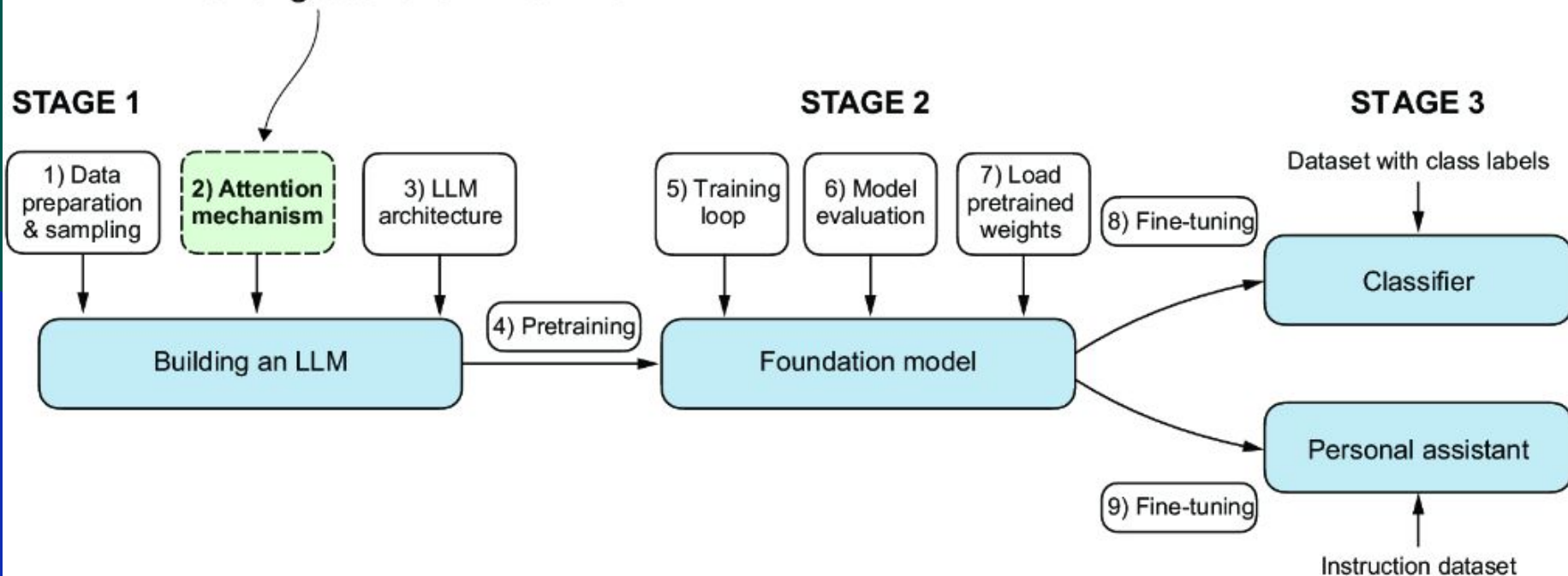
Chapter 3: Coding Attention Mechanisms

Yue (Samuel) Yu



Summary

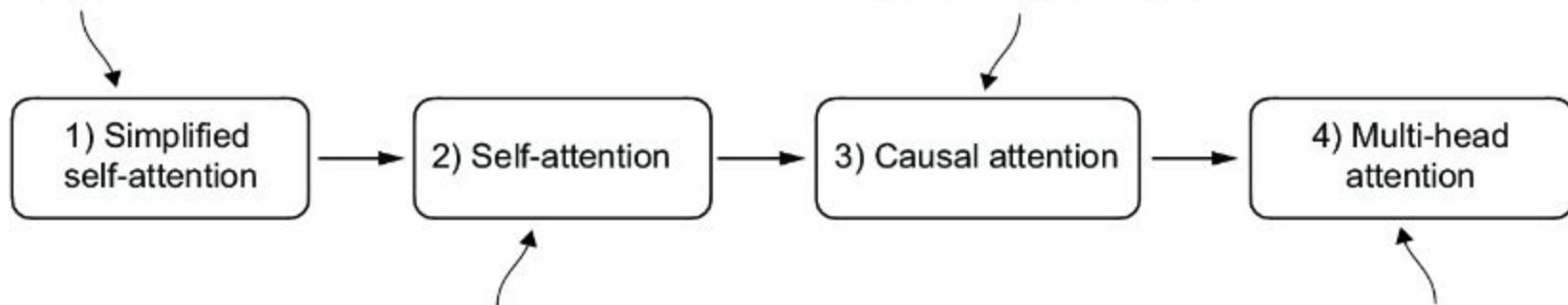
This chapter implements the attention mechanism, an important building block of GPT-like LLMs



Summary

A simplified self-attention technique to introduce the broader idea

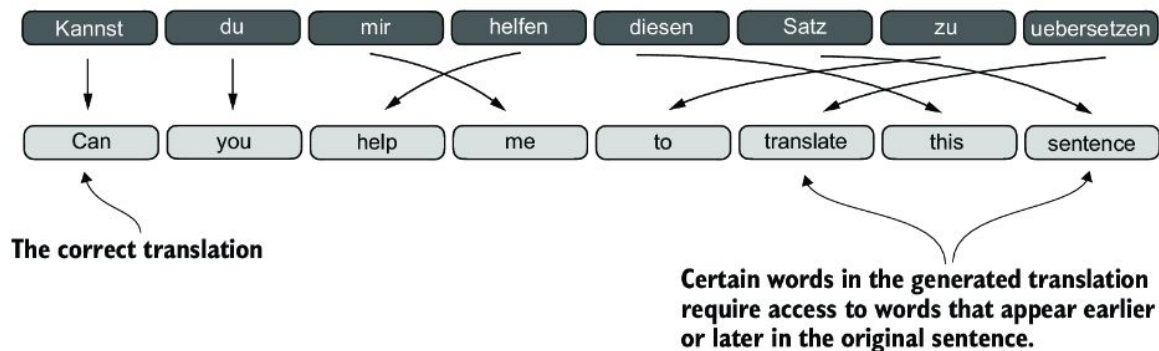
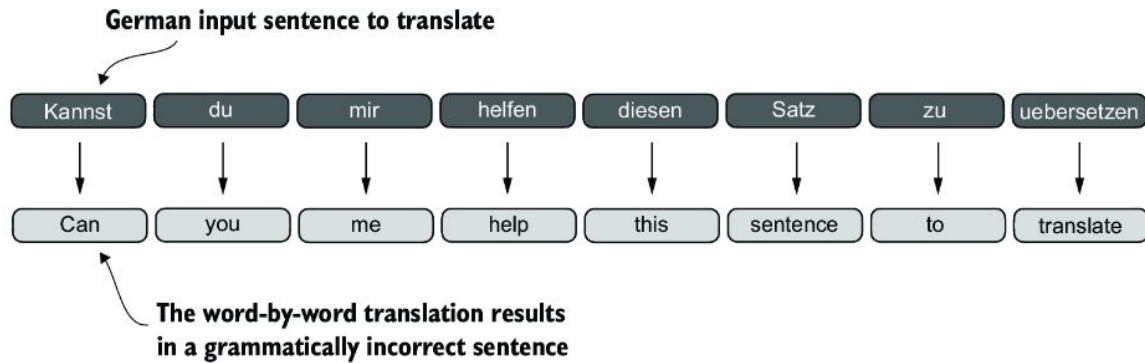
A type of self-attention used in LLMs that allows a model to consider only previous and current inputs in a sequence, ensuring temporal order during the text generation



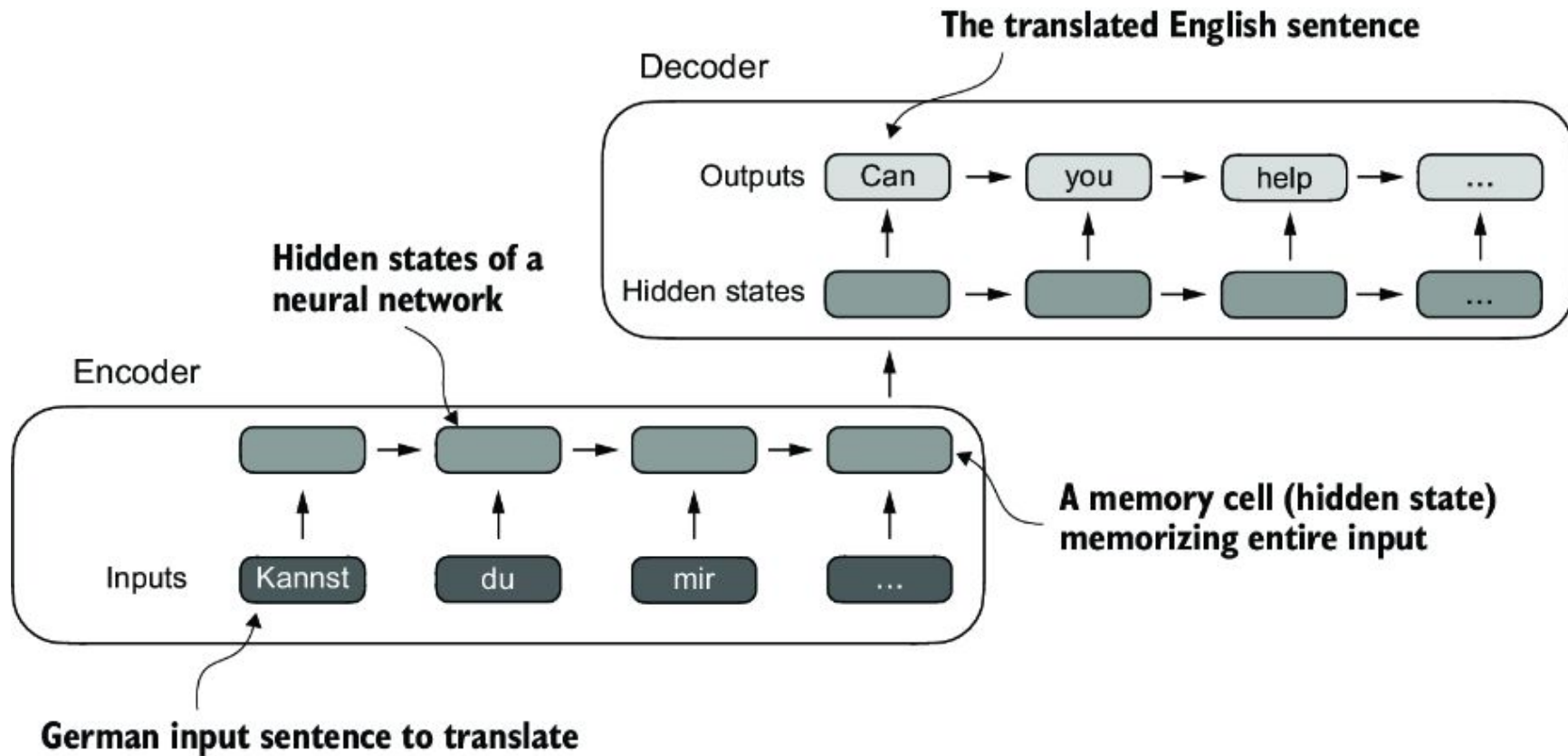
Self-attention with trainable weights that forms the basis of the mechanism used in LLMs

An extension of self-attention and causal attention that enables the model to simultaneously attend to information from different representation subspaces

3.1 The problem with modeling long sequences

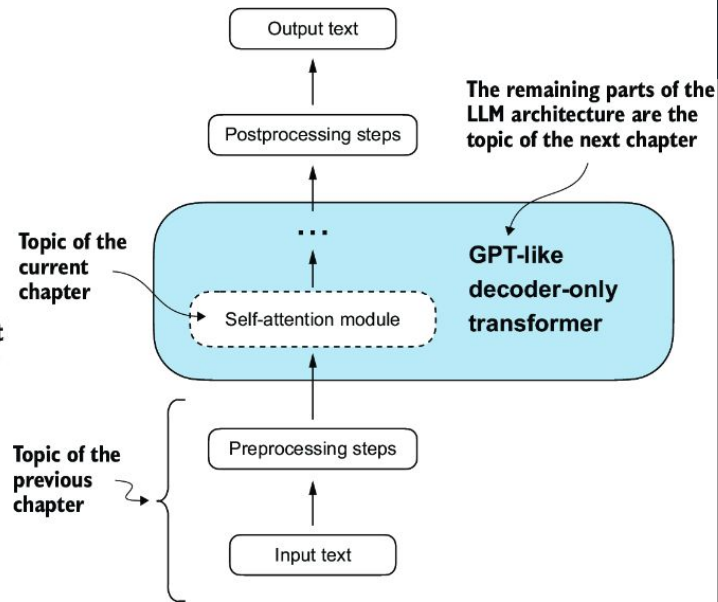
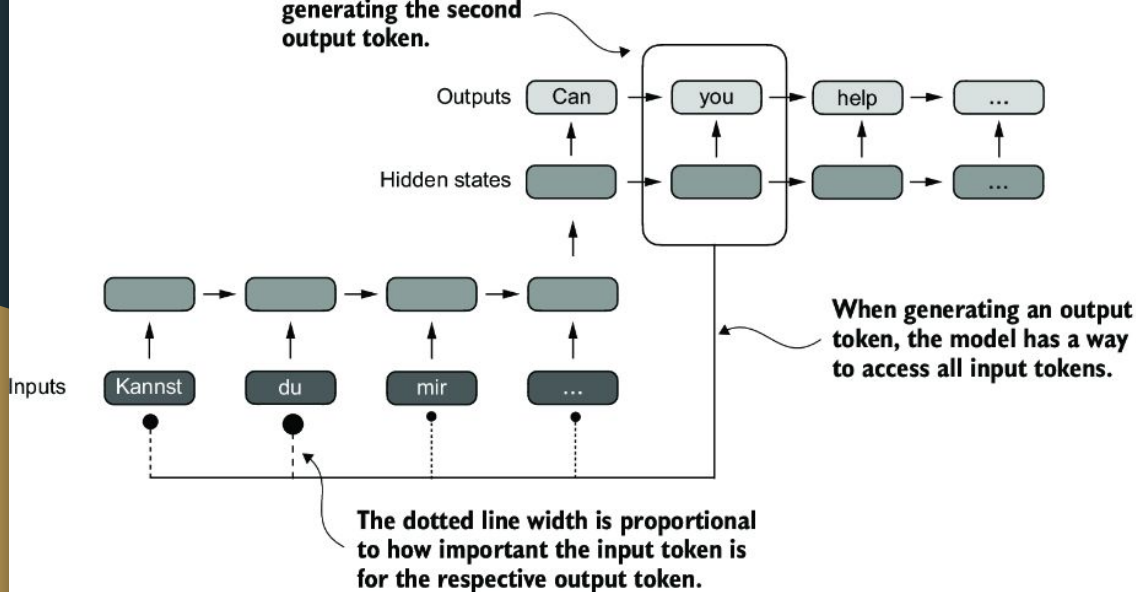


3.1 The problem with modeling long sequences

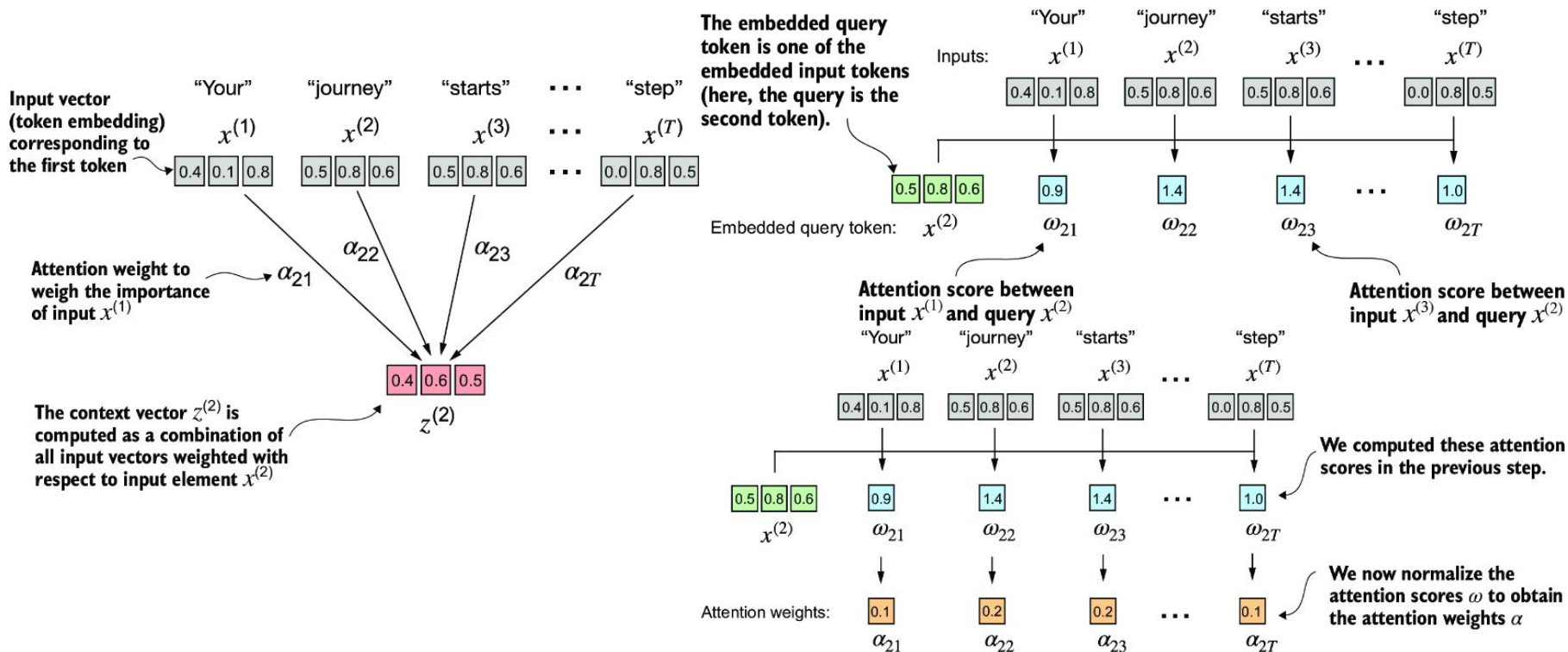


3.2 Capturing data dependencies with attention mechanisms

We are focusing on generating the second output token.



3.3.1 A simple self-attention mechanism without trainable weights



	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

This row contains the attention weights (normalized attention scores) computed previously

3.3.2 Computing attention weights for all input tokens

1) Compute attention scores

Compute the attention scores as dot products between the inputs.

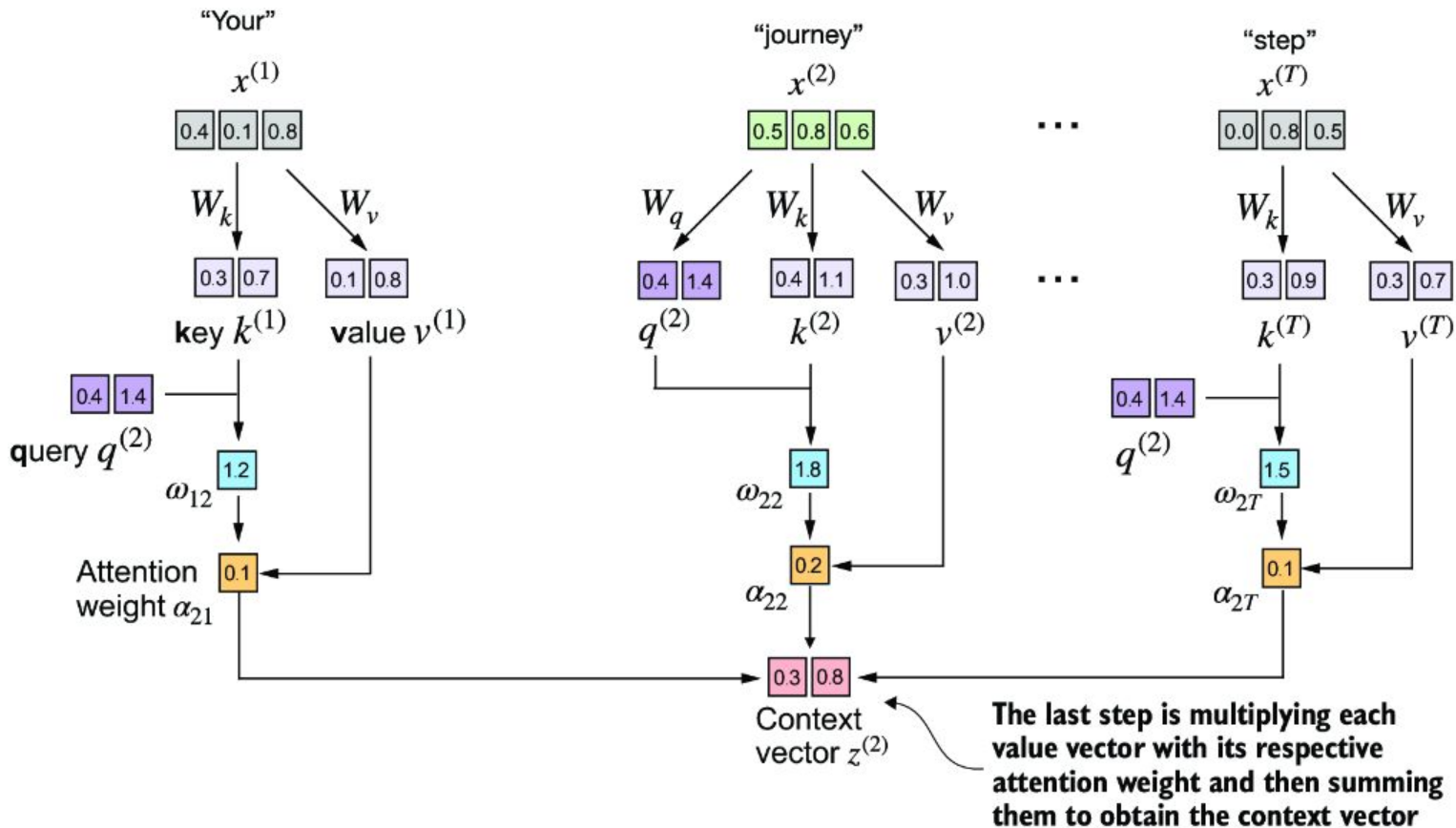
2) Compute attention weights

The attention weights are a normalized version of the attention scores.

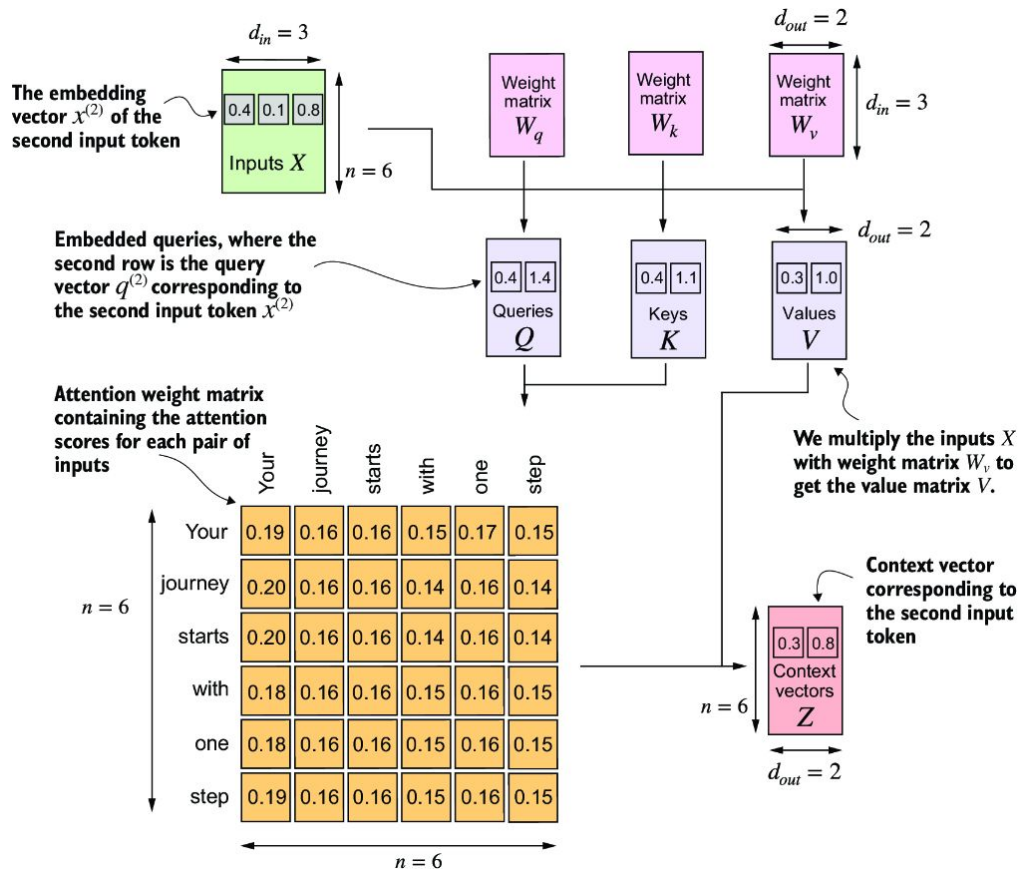
3) Compute context vectors

The context vectors are computed as a weighted sum over the inputs.

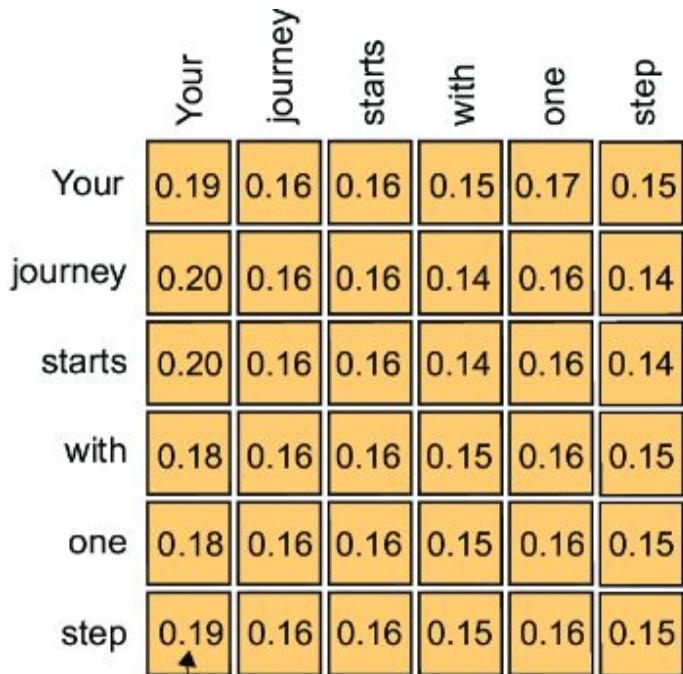
3.4.1 Computing the attention weights step by step



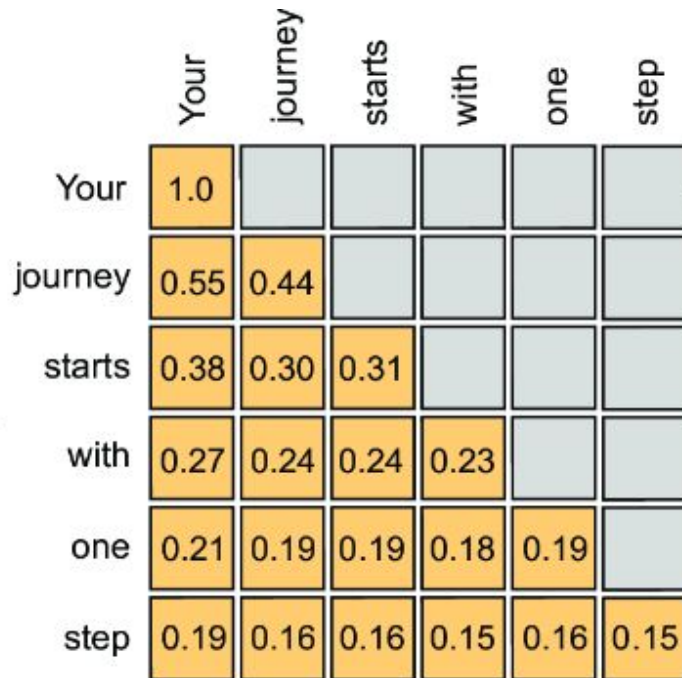
3.4.2 Implementing a compact SelfAttention class



3.5 Hiding future words with causal attention

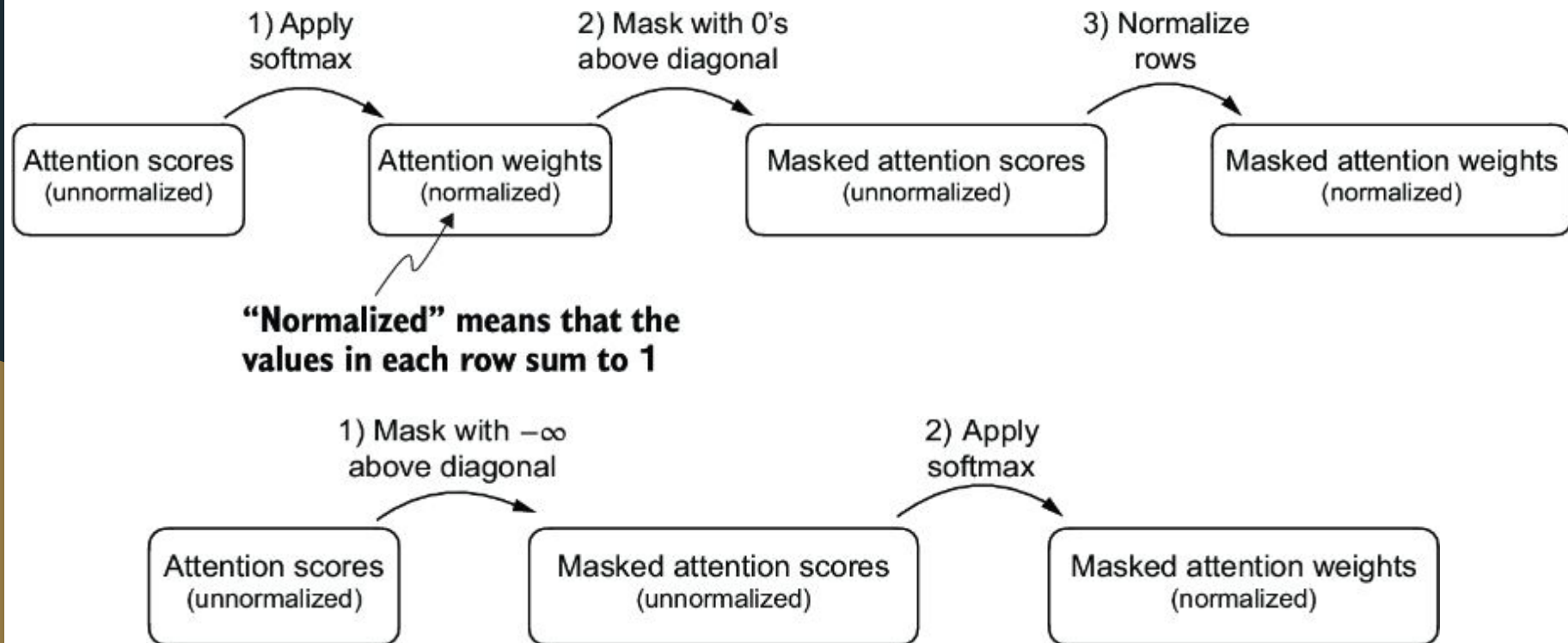


Attention weight for input tokens corresponding to "step" and "Your"

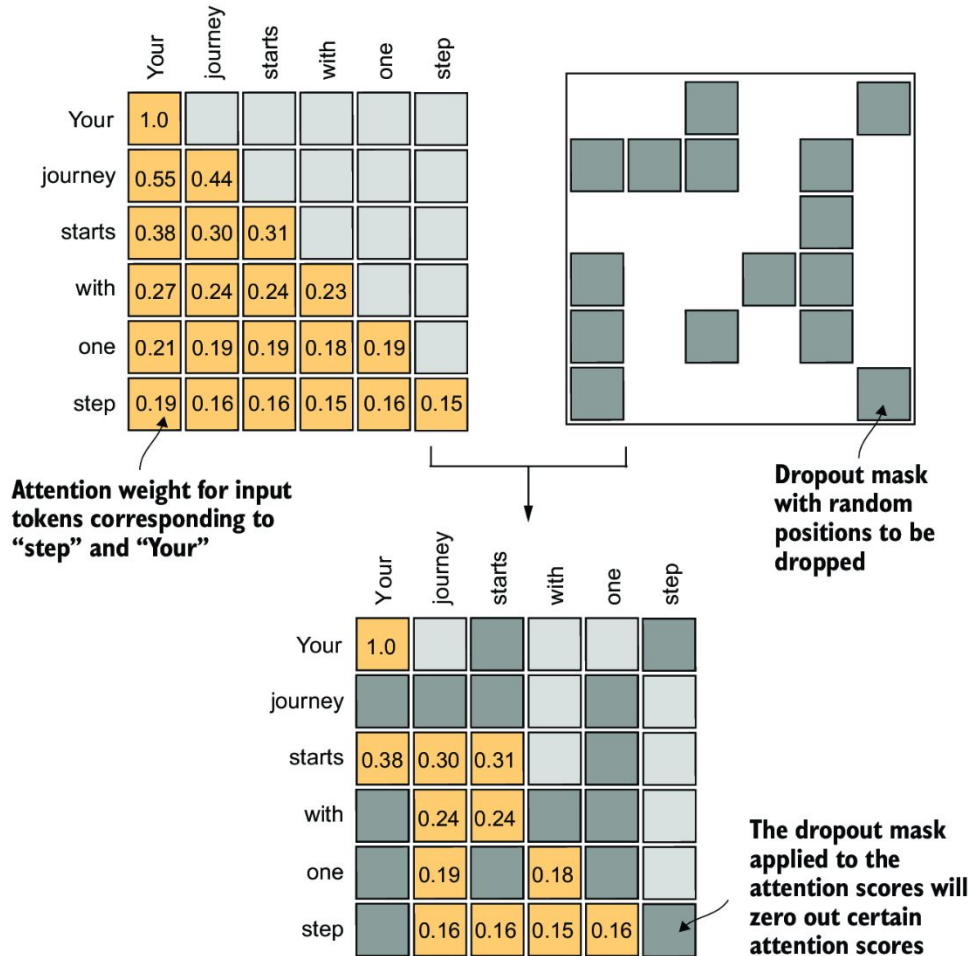


Masked out future tokens for the "Your" token

3.5.1 Applying a causal attention mask



3.5.2 Masking additional attention weights with dropout

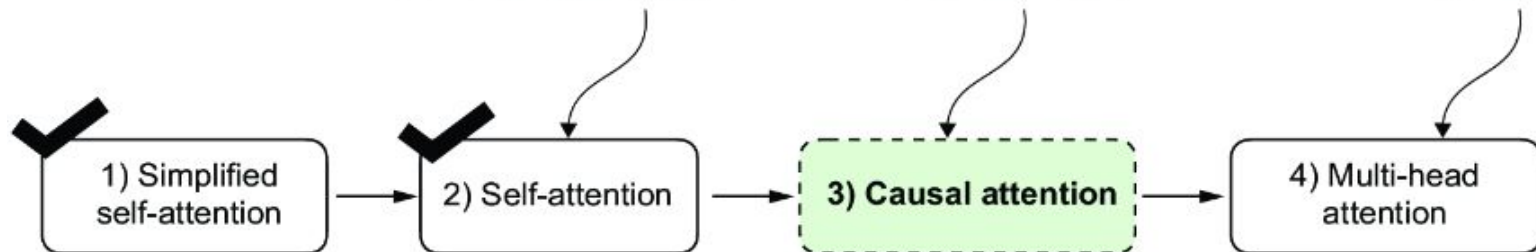


3.6 Extending single-head attention to multi-head attention

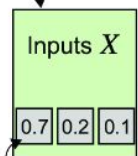
In the previous section, we implemented a self-attention mechanism with trainable weights.

In this section, we extended the self-attention mechanism with a causal mask and dropout mask.

In the next section, we extend causal attention to multi-head attention.



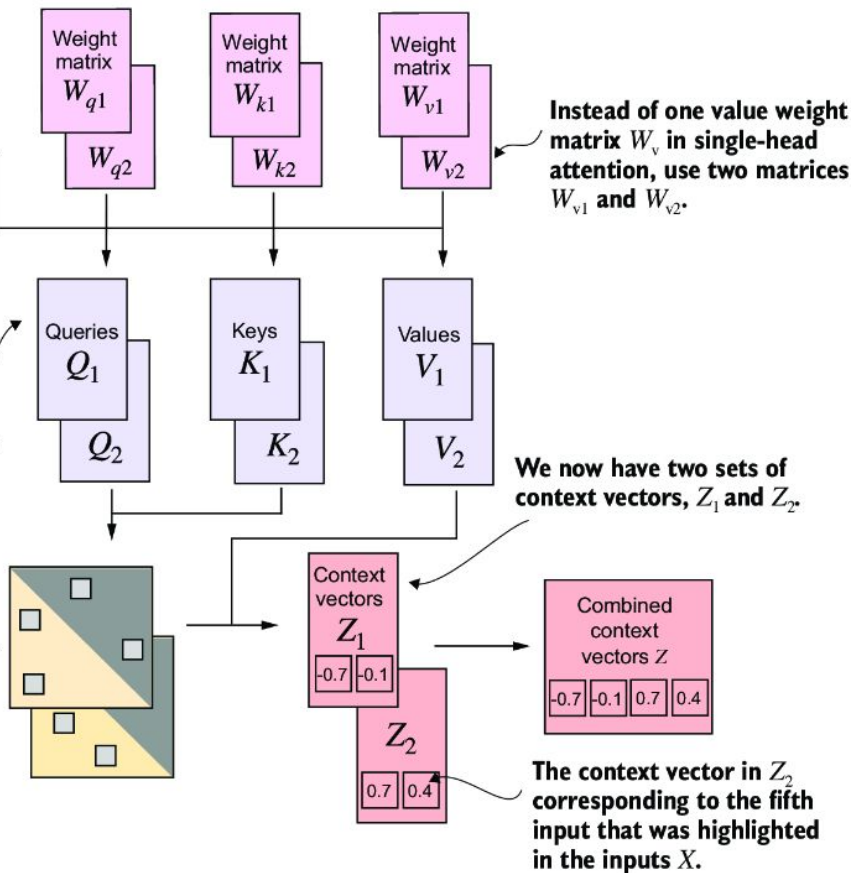
The embedded input tokens remain unchanged.



The values of the 5th row (input) are shown as an example.

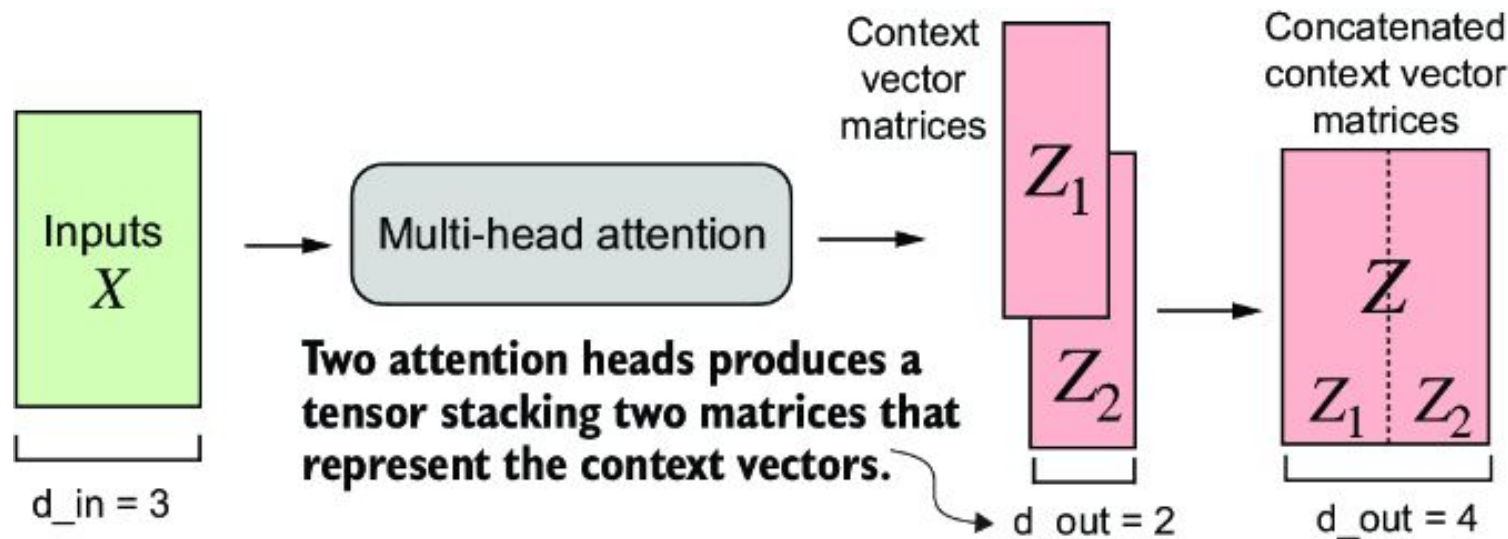
Instead of one query matrix Q , we have two query matrices Q_1 and Q_2 .

For multi-head attention with two heads, we obtain two attention weight matrices, including causal and dropout masks.



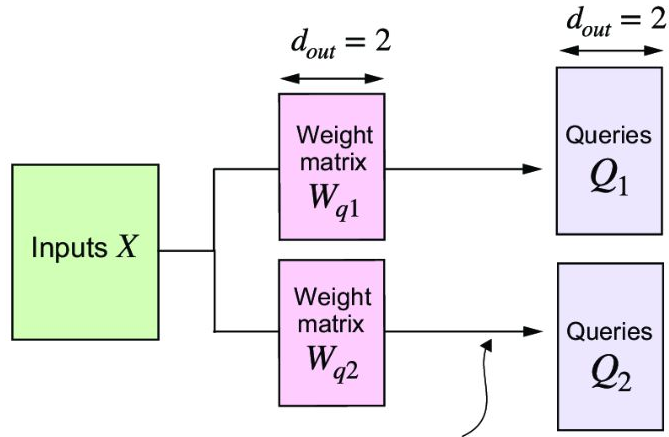
3.6.1 Stacking multiple single-head attention layers

3.6.1 Stacking multiple single-head attention layers

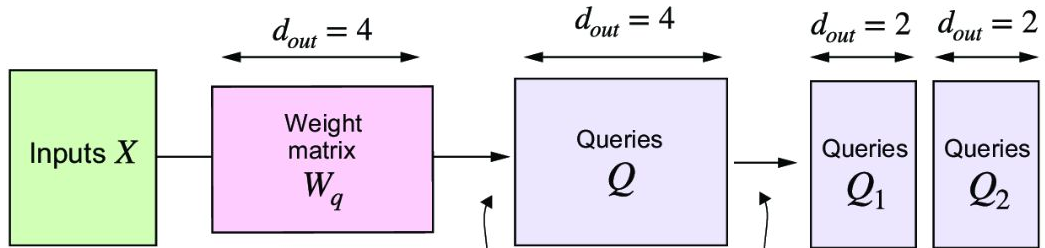


Choosing an embedding dimension of 2 ($d_{out} = 2$) for the context vectors results in a final embedding dimension of 4 ($d_{out} \times \text{num_heads}$).

3.6.2 Implementing multi-head attention with weight splits



Perform two matrix multiplications to obtain the two query matrices, Q_1 and Q_2 .



Obtain queries Q with only one matrix multiplication.

Then split queries Q into Q_1 and Q_2 .